Uniqueness Is Not Continuity: Supplementary Material

Complete proofs, protocols and reproduction

Anonymous Author(s)

Abstract

This document holds the complete statements and proofs behind the four-page paper *Uniqueness Is Not Continuity: When Server Metadata Certifies a Web Snapshot*, the protocols and full results of its executions and replay, and the commands that reproduce every number. Results keep the paper's numbering. Section A maps each claim of the paper to its proof, data and code. Every table of counts is generated from the derived evidence by `scripts/supplement_tables.py`, and every number shared with the paper comes from the same generated macros. The paper does not depend on this document.

CCS Concepts

• **Information systems** → **Digital libraries and archives**; *World Wide Web*; • **Theory of computation** → *Problems, reductions and completeness.*

Keywords

Web archiving, Web data provenance, temporal coherence, HTTP validators, versioned storage

ACM Reference Format:

Anonymous Author(s). 2027. Uniqueness Is Not Continuity: Supplementary Material: Complete proofs, protocols and reproduction. In *Proceedings of The ACM Web Conference 2027 (ACM Web '27).* ACM, New York, NY, USA, 13 pages.

A Overview

The paper asks when equal server metadata at two captures proves that a resource stayed unchanged in between, and when captures of several resources prove that their observed contents were current together. This supplement gives the proofs in the paper's order (Sections B to D), then the evidence: platform semantics, local executions and the documentation audit (Section E), the edit-history replay (Section F), the controlled HTTP executions (Section G) and reproduction (Section H). Table 1 lists where each claim is proved or measured.

Four kinds of evidence are kept apart throughout. A proof holds for the stated model. A finite enumeration checks an implementation against an independent brute-force oracle; it does not replace a proof. A local execution records what one pinned implementation did in one run on loopback; it is not a cloud measurement and estimates no prevalence. Vendor documentation states intended behavior, and the paper's claims about cloud services rest on it alone: no AWS, Google Cloud or Azure service was measured.

The supplement covers only what the paper uses. Results of earlier rounds that the paper no longer prints (finite-capture ambiguity, the unbounded obstruction order, label-alphabet synthesis, a six-page source-history study) are kept, unchanged, in `research/supplement-earlier-rounds.md`; nothing here depends on them.

ACM Web '27, Dublin, Ireland
2027.

B Continuity of one resource

B.1 Contracts, reads and captures

Definition B.1 (Contract). Fix a resource, its request context (including content negotiation) and the authority whose state a read is meant to describe. A *contract* is a finite directed graph $G = (V, E)$ of every server state and transition permitted in that scope, including those of every writer and background process, with a set $R \subseteq V$ of *certifiable* states. Each state q has an authoritative representation $a(q)$, possibly absence $\perp$; a state $q \in R$ answers a read with a body $b(q)$ and fields $h(q)$, its *observation* $o(q) = (b(q), h(q))$. Every finite path of G is a permitted history, and every state may persist for any positive time.

States outside R stay in G. With R the successful current-object reads, the path $A/g \to \perp \to A/g$ still breaks continuity although its 404 state supplies no certified observation; conversely $\perp \to A/g \to \perp$ imposes nothing on its endpoints, which are not certifiable. $R = \emptyset$ certifies nothing, and $R = V$ is the all-state special case.

Read fidelity requires $b(q) = a(q)$ for every $q \in R$. A cache can replay an earlier origin response without describing the origin's current state; such a read fails fidelity whatever its fields, which is a different failure from an intervening return. If the claim concerns the cache itself, a must describe the cache.

A *capture* is an atomic read whose instant lies in a known window $[s, e]$, $s < e$, of one monotone client clock. Captures are *passive*: they do not change the contract state, so the permitted paths do not depend on when or whether anyone reads. This is the model of a Web client reading through public interfaces; it does not claim that a real server keeps no per-request state, only that no such state constrains the publication paths the contract permits.

For an observation ℓ with body x let $S_\ell = \{q \in R : o(q) = \ell\}$ and $P_x = \{q \in V : a(q) = x\}$. Reach and CoReach are taken in the complete graph G, not in the subgraph induced by R, and include paths of length zero.

B.2 The continuity criterion

Theorem 1 (Continuity criterion). *Captures returning ℓ and later ℓ' prove, for every permitted history, that the representation is x from the first read instant to the second iff*

$$\mathrm{Reach}(S_\ell) \cap \mathrm{CoReach}(S_{\ell'}) \subseteq P_x \quad \text{in } G.$$

Proof. *Sufficiency.* A permitted history is in some state of S_ℓ at the first read and in some state of $S_{\ell'}$ at the second; every state it passes in between lies on a path from S_ℓ to $S_{\ell'}$, so it is reachable from S_ℓ and co-reachable to $S_{\ell'}$, hence in P_x by the inclusion. *Necessity.* Let z lie in the intersection with $a(z) \neq x$. A path from S_ℓ to z

Table 1: Where each claim of the paper is proved or measured.

Paper	Claim	Here	Evidence, code and data
Theorem 1	Continuity criterion and its certificates	§B	Proof; product search and invariant checker in `src/history_contracts.py`; 13 constructed models and 37,888 exhaustive contracts against an independent oracle
Proposition 2	Sound fields, no finite path-preserving repair, identity non-reactivation	§C	Proof; `repair()` and `refine()`; 28,864 contracts against all labelings, 2,000 finite refinements, 32 bounded S3 epochs
§2, §4	Documented operations of Amazon S3 and other platforms; local probes; audit	§E	Vendor documentation; 445 loopback requests on nine implementations; two MinIO executions (51 and 60 requests); 39 identifiers from 106 sources
Theorem 3, Corollary 4	Exact test, schedules, fixed instants	§D	Proof; `src/certification.py`; schedules and window sets against brute force
Proposition 5	Worst-case sweep, coNP-completeness	§D	Proof; sweep against the reference semantics; 144 CNF instances of the reduction
Figure 1	Recorded MinIO execution; revisit order	§E, §D	`figure-artifacts/05-certification-rule`
Table 1	Identifiers across a change and its undo	§E	`table-artifacts/01-identity-probes`
Figure 2, §4	Edit-history replay	§F	Protocol committed before collection; two labelled post hoc addenda; `experiments/revision_bundles`, `figure-artifacts/06-revision-histories`
§1, §2	Validators from restored modification times repeat (HTTP controls)	§G	Six configurations on macOS and on Linux; `scripts/run_stack_experiments.py`

followed by a path from z to $S_{\ell'}$ is a finite path, and since every finite path fits into any positive interval with positive holding times, some permitted history follows it between two read instants and has representation $a(z) \neq x$ in between. If z is itself an endpoint, that endpoint read is unfaithful. □

A field is *sound* for a target if the criterion holds for every pair of equal observations of that target, *reusable* if after every capture the resource can leave the target and return to the same observation, and *general* otherwise. Content hashes are reusable under ordinary writes, and so are S3 version IDs under the documented deletions (Section E).

B.3 Certificates

Counterexamples. For each observation ℓ with body x, the product graph has states (q, d) with a sticky departure bit d: from (q, d) an edge $q \to u$ leads to $(u, d \vee a(u) \neq x)$. The initial states are (q, false) for $q \in S_\ell$ and the accepting states (q, true) for $q \in S_\ell$. An accepting product path is exactly a path of G that starts and ends with observation ℓ and visits another representation, so a multi-source breadth-first search returns a shortest violation for ℓ, and the minimum over observations is the shortest overall. With $2|V|$ product states and $2|E|$ product edges per observation, k observations take $O((k+1)(|V|+|E|))$ time including validation; fidelity failures, violations of length zero, are checked first. A separate checker verifies a returned path edge by edge: both endpoints in R, equal endpoint observations, faithful endpoints, and an intermediate state with another representation. It establishes validity; shortestness follows from the search and is cross-checked against an independent all-pairs oracle.

Invariants. When no violation exists, the solver returns for each observation $\ell = (x, z)$ two sets B_ℓ and D_ℓ of states reachable before and after a departure (they may overlap). The checker verifies, without search: one certificate per observation occurring on R; $S_\ell \subseteq B_\ell$; $a(q) = x$ on B_ℓ; every edge $q \to u$ with $q \in B_\ell$ leads into B_ℓ if $a(u) = x$ and into D_ℓ otherwise; D_ℓ is closed under edges; $D_\ell \cap S_\ell = \emptyset$; and fidelity on R. By induction on path length every path from S_ℓ is represented with its correct departure bit, so no path returns to S_ℓ after a departure, self loops and arbitrary repetitions included. The reachable product states always form such a certificate, so the format is complete for the finite model.

Bounded transitions. Let d be the length of a shortest violation, or ∞ if none exists. If a separately justified bound allows at most K transitions between two reads, continuity holds for every observation exactly when $K < d$: no violation fits, and the shortest one witnesses the converse. The bound counts transitions, not elapsed time or captures. A binary body with a counter modulo 3 needs $d = 6$ changes for a hidden return (equal bodies need an even number, equal counters a multiple of 3); three bodies that always change give $d = 3$; a counter modulo 4 with a binary body gives $d = 4$. A bound derived from counter width alone need therefore not be tight.

B.4 Request windows and bridges

Let two equal certifiable observations of a sound field be read at instants $u \in [s_1, e_1]$ and $v \in [s_2, e_2]$ with $e_1 \leq s_2$. Then $u \leq e_1 \leq s_2 \leq v$, so the resource keeps its representation on the closed *bridge* $[e_1, s_2]$, the largest interval guaranteed for every placement of the reads, since $u = e_1$ and $v = s_2$ are possible and the representation may change immediately outside them; for touching windows it is the single instant $e_1 = s_2$. The helper `certified_read_bridge` returns the half-open bridge $[e_1, s_2)$, the convention of the preregistered replay, and nothing when that is empty, which means only that no bridge of positive length exists, not that a common instant is impossible. The windows must enclose the atomic reads on one monotone client clock; independently clocked observations need certified error bounds first, and the HTTP `Date` and `Last-Modi`

fied headers are not effective times. The helper common_times intersects bridges of several resources; an empty intersection is unresolved rather than negative, because bridges can be shorter than the true presence intervals.

B.5 What a certificate does not prove

A positive certificate proves a property of the supplied graph. It does not prove that the graph contains every deployment transition, that a cache read is linearizable to the origin, or that a restart identifier is globally fresh. If the graph is a conservative abstraction that contains every concrete trace and preserves observations and representations, a positive result transfers to the system; a negative abstract path must be concretized before it shows a real failure. Observing few transitions never justifies removing permitted ones.

B.6 Exhaustive checks

data/constructed/contracts.json holds 13 constructed contracts that each exercise one assumption (Table 2). scripts/run_contracts.py computes each verdict, checks every returned certificate with the separate checker and rejects any disagreement with the declared verdict. Their mix of safe and unsafe models has no frequency interpretation; the three fidelity failures are not hidden returns.

The unit tests compare the implementation with an independent oracle on every directed graph on three named states with optional self loops ($2^9 = 512$ graphs), each with every binary body and validator assignment (8×8): 32,768 contracts with $R = V$. The oracle computes all-pairs shortest distances by Floyd–Warshall and searches endpoint pairs with equal observations and an intermediate state with another body; verdicts and shortest lengths agree everywhere, and every certificate passes the checker. A further 5,120 contracts enumerate certifiable scopes: every two-state digraph with binary representations and fields and every R (1,024), and every ordered three-state acyclic graph likewise (4,096), with response bytes outside R deliberately different from the representation. These are correctness checks of finite instances, not benchmarks.

C What a field can certify

C.1 Statement

Let $G = (V, E)$ be a contract with certifiable states R and representations a, and assume read fidelity on R. A *field of the state* is any function f on R; its observation at q is $(b(q), f(q))$, and it is sound if for all $q, q' \in R$ with equal observations every path from q to q' stays in $P_{a(q)}$.

Definition C.1 (Finite refinement). A *finite refinement* of G is a finite contract $G' = (V', E')$ with a surjective map $\pi : V' \to V$ that sends every transition of G' to a transition of G and lifts every path: for every $v' \in V'$ and every path of G that starts at $\pi(v')$, some path of G' starts at v' and maps onto it. A state v' inherits the representation, body and certifiability of $\pi(v')$; its fields may also expose the added state.

A counter, a log of recent operations or any other bounded memory that the server keeps without forbidding an operation gives a finite refinement. Because captures are passive, a reader cannot constrain which path the refinement takes.

Proposition 2 (What a field can certify). *Assume read fidelity.* (i) *Some field computed from the state is sound iff every strongly connected component of G that contains a state of R has one representation.* (ii) *If none is, none is in any finite refinement of G, whereas an unbounded count of representation changes always is.* (iii) *In a store of immutable versions whose current reads return $(\beta(v), v)$, equal identities certify continuity for every payload assignment β iff no path from a state whose current identity is v back to such a state passes another current identity or none; a field recoded statelessly from the returned fields fails whenever they do.*

C.2 Proof

(i) Suppose a component C contains $q \in R$ and z with $a(z) \neq a(q)$. Paths $q \rightsquigarrow z$ and $z \rightsquigarrow q$ exist, and together they form a closed walk from q to itself through z. Any field of the state takes the same value at both ends, which are the same state, and the bodies agree, so the two observations are equal while the walk leaves $a(q)$: no field of the state is sound. Conversely, suppose every component that contains a certifiable state has one representation, and let $f(q)$ be the index of q's component. If $q, q' \in R$ have equal observations they lie in one component C, and a path from q to q' cannot leave C: a state w on it is reachable from q and reaches q', which reaches q, so $w \in C$. Every state of the path has representation $a(q)$, and f is sound. Components take $O(|V| + |E|)$ time; repair() in src/history_contracts.py returns either the component labels, which the independent checker of Section B accepts as a sound field, or a closed walk through another representation. It makes one pass over the states to record, per component, a representative and one state of another representation, and a second pass over the certifiable states, so it is linear; it is tested on a chain of 20,000 states.

(ii), impossibility. Let W be the closed walk of (i) from q through z. Choose a lift q_0 of q (π is surjective). Lifting W from q_0 gives a walk of G' that passes a lift of z and ends at a lift q_1 of q; lifting W again from q_1 gives q_2, and so on. As V' is finite, $q_i = q_j$ for some $i < j$, and the lifted walks from q_i to q_j form a closed walk of G' through a lift of z. The lift q_i is certifiable with $a(q_i) = a(q) \neq a(z)$, so its component in G' mixes representations, and by (i), whose proof does not use finiteness, no field of G' is sound. Conversely, a closed walk of G' maps to a closed walk of G, so every refinement of a contract that satisfies (i) satisfies it too.

(ii), unbounded repair. Refine G by a counter $c \in \mathbb{N}$ that increases at every transition that changes the representation and never decreases, and let the field be c. Equal counts at two reads leave no room for a change between them, so the field is sound. The refinement is infinite, as (ii) requires; no bound on c is consistent with permitting every path of G when G has a closed walk through a change.

(iii) If a path from a state with current identity v back to such a state passes $w \neq v$, assign $\beta(w) \neq \beta(v)$; a state with no current identity is absent, which differs from every payload. Otherwise the current identity, and with it the payload, stays v between equal identities. Equal fields h give equal recodings $g(h)$, so every violation persists after a stateless recoding; a client that keeps state,

Table 2: The constructed contracts: verdicts and shortest violations, in transitions.

Model	States	Edges	Verdict	Shortest violation
`content_hash_return`	2	4	hidden return	2
`modulo_4_counter`	8	16	hidden return	4
`epoch_reset_on_restart`	3	5	hidden return	2
`boot_nonce_reused`	3	5	hidden return	2
`unique_boot_nonce`	3	5	safe	–
`durable_epoch`	3	5	safe	–
`hash_with_no_return_policy`	2	3	safe	–
`cached_old_response`	3	5	unfaithful read	0
`lagging_replica`	3	5	unfaithful read	0
`non_atomic_body_token`	3	5	unfaithful read	0
`irrelevant_internal_transition`	2	4	safe	–
`modulo_3_binary`	6	12	hidden return	6
`modulo_3_ternary`	9	27	hidden return	3

reads more often or receives other trusted information has different inputs and is outside the statement.

The condition of (iii) is not necessary for one fixed payload assignment: $A/v \to A/w \to A/v$ changes identity without leaving A. It is sufficient in a concrete operation model that many stores approximate: immutable data and marker records with one time-independent strict total order, a committed set L that transitions only enlarge, and a current read that selects the maximum of L (absence for a marker or an empty set). For $L \subseteq L'$, $\max L \leq \max L'$, so a displaced head never returns. The model is not complete: two paths $a_1 \to b_1$ and $b_2 \to a_2$ with identities a and b never reactivate an identity, yet a global ranking would need $a < b$ and $b < a$.

C.3 Unbounded state, horizons and finite identifiers

Part (ii) leaves two ways out. The first is unbounded state, such as a per-key publication epoch that every change of the current representation advances, that nothing decreases, and that every current read returns. The second is a contract that removes the cycle itself: if the contract bounds the number of changes, for instance within a certification horizon, a bounded counter suffices, because that contract no longer permits every path of G. The constructed models of Table 2 show both sides: the counters modulo three and four admit hidden returns, while `durable_epoch` and `unique_boot_nonce` are safe only because their finite models end after a fixed number of changes.

Production identifiers are finite. The GCS object schema declares `generation` as a 64-bit integer, and MediaWiki revision IDs are integers of a fixed width [12, 14]. By the argument of (ii), a finite identifier can be sound only over a horizon in which it is not reused; a deployment contract has to state that horizon, including what happens at exhaustion, reset or restoration. This does not suggest that these services wrap around in practice; it says what a proof that relies on them must assume.

C.4 Relation to shared memory

In shared memory, change can be detected with bounded objects because readers take part. Afek et al. obtain bounded snapshots by letting each scanner and updater exchange handshake bits, "each written by one and read by the other" [1], and the lower bounds of Aghazadeh and Woelfel on ABA-detecting registers built from bounded objects grow with the number of processes [2]. Sulimany, Cory and Petrank show that linearizable adaptive snapshots, whose `Click` takes a snapshot and whose `Observe` reads a component of it, cannot have invisible reads, that is, `Read` operations that perform no write or read-modify-write, under obstruction freedom, component encapsulation, update-linearization independence and an oblivious or k-bounded `Click` [19]. Their object and guarantee differ from ours: we analyze a finite transcript of current reads and ask whether a combination existed in every compatible history, and our bound counts server states rather than accessed components. Their result does not imply (ii), nor (ii) theirs. Both concern reads that leave no trace, in different senses: their reads perform no write or read-modify-write on shared objects, whereas ours leave the modeled contract state unchanged, an assumption about the supplied contract rather than the same property of a deployment. And (ii) is narrower than a general price of passive reads: it applies when a certifiable state can recur through another representation, and it excludes only finite additions of state that preserve every permitted path; a finite design that forbids some paths is outside it. Server-driven consistency [22] keeps per-client leases and epoch numbers so that clients detect lost invalidations and revalidate; it keeps caches current and needs a server that tracks its clients, whereas we ask what a client can prove from fields a server already exposes.

C.5 Versioned stores

The documented S3 operations act on a per-key list of versions and delete markers [4]. PUT appends a version, DELETE with a version ID removes that version, and an unqualified DELETE appends a delete marker, which DELETE with the marker's ID removes again. After PUT B and the deletion of B's version, and after inserting and deleting a marker, the list is what it was before. The modeled state is the key's list and its stored representation metadata; logs, replication state and billing need not return and are not part of the observation, and headers generated per request, such as `Date` or a request ID, are not fields of this state. The state $[A]$ thus lies on closed walks through $[A, B]$ (body B) and through $[A, \mathit{marker}]$

(absence), and by (i) no field computed from it — version ID, ETag, `Last-Modified`, size, checksum, or a recoding of them — certifies continuity; by (ii) no bounded addition helps. Two repairs remain: a per-key publication epoch that these deletions also advance and every current read returns, or a contract in which versioning stays enabled and no principal can delete a version or a delete marker, so that the list only grows. The MinIO profile of Section E shows why the second is fragile: denying version deletion blocked both deletions for a restricted publisher, but Object Lock alone still allowed $A/ID \to \bot \to A/ID$, and the owner bypassed a bucket deny; Object Lock does not protect delete markers [3].

GCS generations and Azure Blob version IDs satisfy (iii) within their documented scopes, since deletion promotes nothing and restoration writes a new generation or version [13, 16]. MediaWiki edits and reverts allocate new revision IDs, but revision IDs are preserved across deletion and undeletion [14], so a page restored in full returns as $A/r \to \bot \to A/r$; the positive scope excludes every administrative deletion path, not only selective undeletion.

C.6 Checks

`tests/test_history_contracts.py` and `scripts/check_certification.py` compare `repair()` with an exhaustive search on every contract with two or three states, binary representations and every non-empty certifiable scope: 28,864 contracts, 18,760 of which admit a sound field. For each, every labeling of the certifiable states (up to renaming) is audited with the independent checker; a sound labeling exists exactly when the component criterion holds, the component labels always pass, and every returned closed walk is checked edge by edge.

For (ii), `refine()` builds the product of a contract with a finite memory whose next value depends on the transition and the current value; every path lifts from every memory value, so the product is a finite refinement. The check draws 2,000 random contracts with two to five states and memories of one to five values (seed 1103) and confirms that the product admits a sound field exactly when the contract does; 657 of the contracts admit none. It also refines the three-state S3 deletion model with epochs counted modulo k and saturating at $k-1$, for $k = 1, \dots, 16$ (32 refinements), none of which admits a sound field, and the unit tests show that an epoch the contract stops after six changes is sound and passes the independent checker.

D Certifying a common instant

D.1 Setting

Resources $1, \dots, n$, $n \geq 2$, have independent finite contracts. Resource i is captured $k_i \geq 1$ times; every capture has a closed window $[s, e]$ with $s < e$ on one monotone client clock. The windows of one resource are ordered and may touch but not overlap: each ends no later than the next one starts. Its captures are sequential, so its true read instants increase strictly, also when two windows share an endpoint. In the *single-run* model of Theorem 3 every capture of i returns the same target observation. The first window of i is $[s_i^-, e_i^-]$ and the last $[s_i^+, e_i^+]$; for $k_i = 1$ they coincide. Each target is *open*: the contract lets it become current just before the first capture and be withdrawn just after the last. Without openness the contract already proves presence outside the captures, which can only help.

The captures *certify a common instant* if every history consistent with the contracts and the captures has an instant at which every target is present: $\forall H\, \exists t$. A *fixed* common instant is one instant, determined by the capture record, at which every target is present in every consistent history: $\exists t\, \forall H$.

Example D.1. Let A have a sound field and be read in $[1, 2]$ and $[5, 6]$, and let B be read once in $[3, 4]$. Every history has a common instant, at B's read; but B may be present only near its read instant, which can lie anywhere in $[3, 4]$, so no instant is fixed. `certify()` returns certified and `fixed_instant()` returns none.

Certification is Cooper and Marzullo's *definitely* modality [7] for the conjunctive predicate "every target is present". With interval timestamps it corresponds to Stoller's Def modality over the "definitely occurred before" order [18]; his Inst modality asks instead for one global state whose local states definitely overlapped. For conjunctive predicates over observed local states, Garg and Waldecker decide *definitely* in polynomial time [9], and for two distinct processes Stoller's conditions take the pairwise form of Theorem 3(i). What differs here is that the local intervals in which a target is present are never observed: a resource is seen only at its captures, and Theorem 1 decides from the contract whether it stayed in between.

D.2 The exact test

Theorem 3 (Exact test). (i) *If every field is sound, the captures certify a common instant iff $e_i^- \leq s_j^+$ for all $i \neq j$.* (ii) *If exactly one resource r has a reusable field and the others are sound, they do iff the others satisfy (i) among themselves and some capture window of r lies in $[L, H]$, where $L = \max_{i \neq r} e_i^-$ and $H = \min_{i \neq r} s_i^+$.* (iii) *With two or more reusable fields they never do.*

Proof. *(i), sufficiency.* Let u_i^- and u_i^+ be the true first and last read instants of i. By soundness and fidelity, i is present throughout $[u_i^-, u_i^+]$. For $i \neq j$, $u_i^- \leq e_i^- \leq s_j^+ \leq u_j^+$, and $u_i^- \leq u_i^+$; hence $t = \max_i u_i^- \leq \min_j u_j^+$ lies in every presence interval. *Necessity.* If $e_i^- > s_j^+$ for some $i \neq j$, read i first at e_i^- and j last at s_j^+, other reads anywhere in their windows; let i become current within δ before e_i^- and j be withdrawn within δ after s_j^+, with $2\delta < e_i^- - s_j^+$ (openness). No instant has both.

(ii). A reusable resource can leave and return between any two of its captures, so its presence can be confined to arbitrarily small neighborhoods of its read instants. If a window of r lies in $[L, H]$, the instant read in it lies in $[\max u_i^-, \min u_i^+]$ over the sound resources, by (i), and is a common instant. Conversely, place the sound resources at the extremes as in (i), so that their certain common interval is exactly $[L, H]$; read r at a point of each window outside $[L, H]$, which exists because no window lies inside it, and confine r to small neighborhoods of those points.

(iii). Confine two reusable resources to neighborhoods of their read instants, choose all those instants distinct, which positive windows allow, and shrink the neighborhoods until they are disjoint. □

The test needs only the two largest e_i^- and the two smallest s_j^+, and a reusable resource's windows are checked in one pass, so it takes time linear in the number of captures (`_violating_pair`). With instantaneous reads ($s = e$) it is the closed-interval intersection that Spaniol et al. use for inducible coherence [17]; what is added is the window model, completeness in both directions, the lower bounds below and the treatment of fields that are not sound. For every refused instance, `counterexample()` builds read instants and presence intervals without a common instant, and `verify_counterexample()` checks them independently. Each reusable read is chosen among $\sum_i k_i + 1$ equally spaced rational points of its window, so an unused instant always remains; an earlier version offered five fixed points and failed on six reusable resources in one window, which is now a regression test, with up to twelve reusable reads checked.

Touching windows. The proofs use the windows of one resource only through $u_k \le e_k \le s_{k+1} \le u_{k+1}$ for consecutive captures $k, k+1$, which touching windows satisfy. The refutations place a read at a window endpoint; where that endpoint is shared with the neighboring window of the same resource, the neighboring read moves into the interior of its own window, which has positive length, so the reads stay strictly ordered and the refutation is unchanged. `counterexample()` does this, and the audit compares the test with brute force on sequential schedules whose windows touch and on random touching window sets (Table 3). The replay's 1 s condition, in which consecutive windows touch, therefore lies within Theorem 3: a closed bridge $[e^-, s^+]$ may be a single instant, and with sound fields that instant is common to every history.

D.3 Schedules and fixed instants

Corollary 4 (Schedules). (i) *Certifying $n \ge 2$ resources needs at least $2n-1$ captures; the order $1, \dots, n-1, n, n-1, \dots, 1$ attains this, and its singly read resource n needs no sound field.* (ii) *A fixed common instant needs every resource captured at least twice, hence $2n$ captures, and no reusable field; with sound fields the fixed instants are exactly $[\max_i e_i^-, \min_i s_i^+]$.* (iii) *With parallel requests, two rounds of $2n$ captures or three rounds of $2n-1$ suffice.*

Proof. *(i)* Two singly captured resources i, j would need $e_i \le s_j$ and $e_j \le s_i$, so $s_i < e_i \le s_j < e_j \le s_i$, a contradiction; hence at least $2(n-1)+1$ captures. In the revisit order no first window ends after another resource's last window starts, and resource n is read once, so Theorem 3(ii) admits a reusable field there; a singly read resource of any class is present at its own read, so the order tolerates a general field at the center as well.

(ii) Let i be captured once in $[s_i, e_i]$ and let t be any instant. Choose i's read instant in the window but different from t, and let i be present only around it (openness): t is not common. A reusable resource is handled alike however often it is captured. Conversely, with every field sound and $k_i \ge 2$, each $t \in [L, H]$ with $L = \max_i e_i^-$ and $H = \min_i s_i^+$ satisfies $u_i^- \le e_i^- \le L \le t \le H \le s_i^+ \le u_i^+$ for every i, so t is common in every history. For $t < L$, the resource with $e_i^- = L$ can be read at e_i^- and appear just before it, after t; symmetrically for $t > H$. In the double collect $1, \dots, n, n, \dots, 1$ with sequential windows, L is the end of window n and H the start of window $n+1$, so $L \le H$, with equality when the two windows touch; when every resource is revisited, Theorem 3(i) is equivalent to $L \le H$, so certifying and fixing coincide.

(iii) Two rounds (all first captures, then all last captures) use $2n$ captures; three rounds ($n-1$ first captures, the single capture, $n-1$ last captures) use $2n-1$. Two rounds cannot use $2n-1$: the single capture would have to lie strictly between them. □

Spaniol et al. introduced the inverted revisit order, with the last visited page downloaded once, under instantaneous downloads, and described ETag or hash comparison as "the only 100% reliable method" [17]. The corollary shows that this order is optimal, and Theorem 3 that it still certifies under request windows whenever the revisited resources have sound fields, while the singly read resource needs no continuity guarantee at all. Part (ii) prices the saved capture: the revisit order places the common instant inside the center's window, at an instant that varies between histories, whereas a double collect [1] of sound fields with closed bridges ($\max e^- \le \min s^+$) names an interval that every history shares. The preregistered replay protocol used half-open bridges ($\max e^- < \min s^+$), which reject a resource whose consecutive windows touch although one common instant suffices; Section F reports both.

D.4 Worst cases and hardness

In the general model the captures of a resource may return different observations, all with the target body. For one consistent history of a resource, its *departure pattern* records whether the resource can be absent before its first capture, whether its path leaves the target in each gap between consecutive captures, and whether it can leave after the last capture. A *worst case* is a consistent history whose pattern dominates the pattern of every consistent history componentwise. Under openness the first and last entries always depart; `departure_pattern()` also handles contracts that are not open.

Proposition 5 (Worst cases and hardness). *Let the captures of a resource return any observations of its target. If every resource has a worst-case history, certification is decidable in polynomial time; this includes the single-run sound and reusable cases of Theorem 3 and every contract whose observations identify their states. In general, deciding certification is coNP-complete, even when every field is sound, contracts are acyclic and windows are arbitrarily short.*

The sweep. Suppose every resource has a worst case. Its *runs* are the maximal blocks of captures joined by gaps that the worst case does not leave. A run of captures $a, \dots, b$ with $a < b$ is certainly present on $[e_a, s_b]$; a run of one capture is present only at its read instant, which may lie anywhere in its window.

Lemma D.1. *Certification holds iff* (A) *the fixed runs of all resources share a point, or* (B) *some resource has a one-capture run whose window lies inside the union of fixed runs of every other resource.*

Proof. *Sufficiency.* A gap that the worst case does not leave is left by no consistent history, since the worst case dominates them all, so in every history the resource is present on $[u_a, u_b] \supseteq [e_a, s_b]$. Under (A) a shared point is common to every history; under (B) the one-capture run's read instant lies in every other resource's

fixed runs. *Necessity.* If neither holds, let every resource follow its worst case, read each run first at e_a and last at s_b, read every one-capture run at a point of its window outside the other resources' common fixed runs and distinct from all other reads, and let each departing gap be absent except in small neighborhoods of its two reads. Presence is then each resource's fixed runs plus neighborhoods of points, and no point lies in all of them: a fixed point fails (A), and a read point fails (B) or meets no other read. Finitely many compact sets with empty intersection keep it empty after the neighborhoods shrink. □

`worst_case_pattern()` computes, by forward and backward reachability over the capture layers, whether some consistent history leaves the target in each gap; the componentwise maximum is the only possible worst case, and one more layered search decides whether one history realizes it. A two-layer search over (state, departed) from every state gives reachability with and without a departure, so a resource with k captures takes $O(|V|(|V|+|E|)+k|V|^2)$ time. `sweep_certainty()` then decides (A) in one sweep over sorted run endpoints, $O(M \log M)$ for M captures, and (B) with one binary search per one-capture window and other resource, $O(nM \log M)$. Sound and reusable single runs have a worst case (no departing gap, respectively all gaps departing in one history), and so does every contract whose observations identify their states, because its gaps are then independent.

Membership in coNP.. A certificate of non-certification gives, for every capture j of resource i, a state q_{ij} that emits the recorded observation. The verifier checks emission, target membership, reachability of q_{i1} from an initial state and of $q_{i,j+1}$ from q_{ij}. Restricting the consistent histories to those that pass q_{ij} at capture j fixes the endpoints of every gap, so paths in different gaps no longer constrain each other: a departing path is chosen in each gap where one exists, every finite path is permitted and holding times have no lower bound, so these choices concatenate. The restricted instance therefore has a worst case, and the verifier runs the sweep on it and accepts iff the sweep finds no certain common instant. A history without a common instant yields an accepted certificate, and an accepted certificate yields such a history by Lemma D.1. The certificate has $O(K \log |V_{\max}|)$ bits for K captures.

Hardness. Reduce from CNF satisfiability with clauses $c_1, \dots, c_q$ over variables $v_1, \dots, v_m$. A clock resource Z has states $f_0 \to z_1 \to f_1 \to \dots \to z_q \to f_q$ with bodies alternating off and on, a fresh observation per z_k and initial state f_0; it is captured at $4k$ and $4k+2$ with observation z_k, so it is certainly present exactly on $[4k, 4k+2]$. Each variable resource X_v has two disconnected branches T and F with identical observations $r_0, \dots, r_q$, captured at $4k+3$. In branch T, the segment between r_{k-1} and r_k also has a departing path through an off state if $v \in c_k$; in branch F, if $\neg v \in c_k$. If the formula is satisfiable, choose branches by a satisfying assignment and in each slot let a variable of a true literal depart across the whole slot; Z covers every other instant, so no common instant exists. Conversely, a history without a common instant fixes one branch per variable, and since Z is present on every slot, in every slot some variable departs, which its branch allows only if its literal satisfies the clause. All contracts are acyclic, and every window may have any width up to $1/2$, down to a point.

Every field in the construction is sound by Theorem 1, independently of how often it is captured. Each clock observation z_k occurs at one state, which cannot be left and re-entered. Each variable observation r_k occurs at most once on each acyclic branch, a path cannot return to it after leaving, and the branches are disconnected, so no path reaches the equal observation on the other branch; no path between equal observations leaves and returns to the target. The variable resources have no worst case: branch T can leave in the slots of clauses containing v, branch F in those containing $\neg v$, and when both literals occur the two patterns are incomparable. Hardness therefore comes from hidden choice behind identical observations, not from unsound fields. Conjunctive predicates over observed local states are polynomial [9], general predicates are hard to detect [6], and weak conjunctive detection becomes NP-complete once local events are only partially ordered [20]; that result asks whether some consistent global state exists, whereas here every history consistent with independent contracts must contain a real-time common instant, and the uncertainty comes from hidden server states. The sufficient condition of Lemma D.1 and the general hardness do not form a dichotomy: a general field without a worst case can still be certified with more captures. Two resources with a contract that permits one return, read A, B, A, B, A in disjoint windows, have a common instant in every history, since A absent at both B reads would need two departures; the reference semantics certifies this instance, while the sweep returns unknown.

D.5 Verification

`src/certification.py` implements the pairwise test, the fixed-instant test, the counterexample constructor and an independent checker, the worst-case pattern and sweep, an exponential reference semantics for small explicit contracts, and the reduction. `scripts/check_certification.py` writes `data/derived/certification-audit.json`; Table 3 lists its checks. They check the implementation and the proofs; they do not replace the proofs.

The audit also confirms the minimum sequential schedules that certify $n = 1, 2, 3, 4$ resources (1, 3, 5, 7 captures) and that fix an instant for $n = 1, 2, 3$ (2, 4, 6), counterexamples for 2, 6, 9 and 12 reusable reads in one window, and the one-return A, B, A, B, A instance.

E Platforms: documentation and local executions

E.1 S3 version identity

An immutable version's identity and the history of which version is current at a key are different objects. AWS documents two restorations: copying an old version creates a new current version ID, while permanently deleting the current version exposes its predecessor under the predecessor's original ID [4]. The current-key history $A/v_1 \to B/v_2 \to A/v_1$ is therefore permitted although no ID was allocated twice. Deleting a delete marker likewise restores the object below it, and $A/v_1 \to \perp \to A/v_1$ remains a counterexample when only successful reads are certifiable; the absent state stays in the contract. A read that selects `versionId=v1` reads a historical version, not the current key. These are documented operations, not server faults, and neither the distinction between

Table 3: Checks against brute force in the certification audit.

Check	Instances
Sequential schedules, $n = 2, 3$, every sound/reusable assignment, against a search over read instants (286 certified)	5,944
Random overlapping windows, same oracle	4,000
Sequential schedules whose windows touch, $n = 2, 3$, same oracle (286 certified)	5,944
Random window sets in which a resource's windows may touch, same oracle	2,000
Sequential schedules, $n = 1, 2, 3$, against a search for a fixed instant (41 fix one)	5,948
Explicit fresh-token and content-hash machines against the rule	361
CNF instances of the reduction, with positive and point windows	144
Random explicit contracts, sweep against the reference semantics (294 observation-deterministic)	591
Random contracts, worst-case pattern against enumeration	3,000
Small contracts, component criterion against all labelings	28,864
Random finite refinements, Proposition 2(ii)	2,000
Bounded epochs on the S3 deletion model	32

Table 4: Implementations probed, their releases and the requests sent to each.

Implementation	Version	Requests
MinIO	`RELEASE.2025-09-07T16-13-09Z`	51
Ceph RADOS Gateway	`19.2.0`	51
SeaweedFS	`3.97`	53
Versity S3 Gateway	`1.10.0`	51
RustFS	`1.0.4`	51
Scality CloudServer	`9.0.27`	51
LocalStack	`4.9.2`	52
moto	`5.1.15`	51
fake-gcs-server	`1.56.1`	34

a version selector and a condition on the current target [5] nor the ABA phenomenon is claimed as new.

E.2 Identity probes on nine implementations

The probes ask whether reactivation is specific to one server. `experiments/reactivation_probes/` ran one fixed probe sequence on eight implementations of the S3 API and, for contrast, on a GCS emulator, since GCS documents that deleting the live version promotes no older generation (Table 4): 445 loopback requests, every byte recorded, on Linux x86_64 on 2026-11-05. Each server used a fresh temporary store and public fixture credentials. The MinIO binary matches the release pinned for the earlier macOS executions, so it also replicates them across platforms.

All eight S3 implementations made the displaced version ID current again after deleting the newest version by ID, and after inserting and removing a delete marker (with a 404 in between). Over a COMPLIANCE-retained version, seven did the same and refused direct deletion of the retained version (400 or 403); SeaweedFS answered the retained PUT with 500, so that probe did not run. All eight kept the identifier (`null` or absent) across a write of A, B, A with versioning suspended. Copying the old version onto the key gave a fresh ID on seven, and the read after the copy returned A's first ETag again; LocalStack required replaced metadata, and SeaweedFS refused all three request forms. Uploading the old bytes again gave a fresh ID on all eight and repeated A's ETag, and a conditional GET with A's first ETag returned 304. No conditional GET was sent after a copy. The GCS emulator promoted nothing when the live generation was deleted (404), and copy and upload allocated new generations; this agrees with the GCS documentation but is not a GCS measurement. The eight S3 servers are a fixed set, not a sample: they show that reactivation holds across implementations of one API, and their number says nothing about how often any of them is deployed. `derive.py` reclassifies every cell from the raw records, and Table 1 of the paper is generated in `table-artifacts/01-identity-probes/` from a full copy of the traces.

E.3 Two MinIO executions

Figure 1a. `experiments/versioned_objects/data/trace.json` records six cases and 51 signed requests against an unmodified MinIO RELEASE.2025-09-07T16-13-09Z (official binary, SHA-256 checked) on macOS; no cloud account was used. Delete-current gives $A/\alpha, B/\beta, A/\alpha$; copy restoration and re-upload give $A/\alpha, B/\beta, A/\gamma$; inserting and removing a delete marker gives $A/\alpha, \perp, A/\alpha$ (Greek letters are per-case aliases; the opaque IDs stay in the mapping file). A pinned control keeps reading historical A while the current key returns B. The sixth case changes two keys in the order A, B, B, A, reading both stable keys after each completed single-key mutation: the target pair (eligible, 10) is absent in all five stable phases, yet the captures A, B, B, A return matching IDs within each key and induce overlapping bridges. This is the execution of Figure 1a. `derive.py` verifies request and response hashes, selectors, the five phase pairs and the capture order; the dedicated store and single writer are execution assumptions that a transcript cannot prove.

Removing deletions. `experiments/publication_profile/` records eight cases, 60 signed requests and five management commands against the same server and a pinned management client. A non-owner publisher may read current and historical objects, write, and delete without a version ID, but is denied `DeleteObjectVersion` and every configuration change.

The rows of Table 5 are observations within one execution, not independent trials. A first attempt stopped because the harness expected 403 for the retained-object deletion while the server returned 400 with a WORM message; that raw attempt is preserved separately, and the reported run is a fresh execution after the assertion was corrected. Privileged mutation, other writers, lifecycle rules, replication, suspension of versioning and restoration of the bucket or its authority are outside the profile, and preventing every version deletion also prevents reclaiming noncurrent versions.

E.4 GCS generations

For finalized objects at a fixed key in a continuously existing flat-namespace bucket, generations are unique across successive data versions, although their values need not increase; metadata-only

Table 5: The second MinIO execution: whether removing deletions removes reactivation.

Case	Outcome	What it shows
Owner under a bucket deny	DELETE of B returns 204; A's original ID returns	The owner's path is not constrained by that deny
Publisher deletes a data version	403; B stays current	Data-version deletion is blocked
Publisher deletes a marker	403; the current read stays 404	Marker removal is blocked
Publisher copies old A	200 and a fresh current ID	Restoration remains possible, with a fresh ID
Batch version deletion	200 with two per-object AccessDenied errors	HTTP success alone does not classify a batch
Publisher changes configuration	403 for versioning, policy and lifecycle	These changes are blocked
Two keys read in a stable phase	Equal IDs; bridges overlap for 793,750 ns	A positive bundle under the declared contract
Multipart uploads completed in reverse	B then A current, both fresh IDs	One observed order, not AWS's
COMPLIANCE retention	Data deletion refused (400, WORM); marker insertion and removal 204	Retained data still permits $A/ID \to \perp \to A/ID$

updates keep the data generation and change only the metageneration [11]. Deleting the live version leaves no live version rather than promoting an older one, copying a noncurrent version back allocates a new generation, and restoring a soft-deleted object writes a new copy [13]. Once a generation stops being live at the key, these operations cannot make it live again, so equal generations at two faithful current-name reads imply continuous stored bytes in between, over any horizon in which generations are not reused. The claim needs successful full current-name reads without a generation selector, with the generation taken from the same response as the bytes; point fidelity (no cache between client and origin, and a fixed encoding, since `Accept-Encoding:gzip` is needed to prevent decompressive transcoding); a fixed bucket, project and key; and the exclusion of bucket or project restoration, hierarchical-namespace folder renames and appendable objects. It says nothing about authorization, availability or response metadata, and no GCS service was measured.

The official Python client pins versions silently. In `experiments/gcs_selector/`, `google-cloud-storage` 3.17.1 runs against a loopback fixture whose current data change from generation 101 (body A) to 202 (body B). Reusing one `Blob` object for two reads sends `generation=101` on the second and returns the old version; a fresh `Blob` per read returns 202 and B; an explicit selector control behaves as specified. This agrees with the client source [10]: the client stores the response's generation and adds it as a selector to later downloads of the same object. Each current-name observation must therefore use a fresh, unbound object.

E.5 Documented identifier semantics

`research/platform-identifiers-round9.md` and its ledger record 39 identifiers on 24 storage, hosting, content-management, registry, Git and Web-server platforms, with 257 quotations from 106 official sources checked verbatim against copies retrieved on 2026-11-05. Each entry records what a read or management interface exposes and what specific restore, rollback or undelete operations do to it. The audit groups the identifiers by what a change and its undo do to them; its counts describe the inspected interfaces and operations, not how widely each is deployed. 13 identifiers are unique but can be made current again (S3 version IDs; B2 file IDs; DigitalOcean Spaces; Ceph RGW, from its tests; Swift object versioning; SeaweedFS, from source; GCS hierarchical-namespace rename, inferred; deploy and version IDs of Netlify, Vercel, Cloudflare Pages and Firebase; npm dist-tags; MediaWiki revision IDs under deletion and undeletion). 14 derive from content or file attributes (MD5 and SHA-1 ETags, content digests, Git and OCI identifiers, modification-time ETags of nginx, Apache and Express), 2 are not unique (S3 `null`, GCS metageneration) and 4 are unclear or unversioned. The remaining 6 receive fresh values on the restore paths inspected (GCS flat-namespace generations, Azure Blob version IDs, Azure ETags and snapshot promotion, Firebase release IDs, WordPress revision restore). A fresh value on one path does not make a field sound for every current read, and three corrections bound these entries.

MediaWiki. Revision IDs are preserved across deletion and undeletion [14], so a page deleted and restored in full returns as $A/r \to \perp \to A/r$; selective restoration can also make an older revision current after another. Only ordinary edits and reverts, which allocate new IDs, are inside the positive scope.

WordPress. Restoring a revision creates a new one, but the current post's `predecessor-version` link names the latest *remaining* revision row [21]. For authorized edit-context reads and the observation (`content.raw`, `predecessor-version.id`), the sequence $A/r_1 \to B/r_2 \to A/r_3$, followed by deleting r_3 and r_2, returns to A/r_1. This is derived from the official core source, not measured, and it does not claim that the whole JSON response repeats.

Azure snapshots. A snapshot's DateTime names an immutable historical snapshot and is supplied later as a `snapshot` selector; the base-blob URI and a version-selected URI are different reads [15]. Copying a snapshot over the base blob writes a fresh current version and ETag, but the snapshot DateTime is not a publication epoch on an unqualified current read. Azure Blob version IDs are inside the positive scope only for versioned block blobs; page and append blobs, whose updates need not create versions, and point-in-time restore are outside it [16].

Several classifications rest on project source or tests where the prose is silent (Ceph, Swift, SeaweedFS, WordPress, MediaWiki undeletion, nginx), and two on inference (GCS rename, Cloudflare Pages); the paper names only explicitly documented cases. Documentation states intended behavior; Table 1 of the paper tests implementations.

F Edit-history replay

F.1 Protocol and collection

`experiments/revision_bundles/protocol.json` fixed the sampling frames, the 2024 window, the capture schedules, the methods and the ground-truth definitions, and was committed to the repository before any revision metadata was requested; the collection manifest records its SHA-256 (`79c8be55...`), which `derive.py` checks. `collect.py` then queried the English Wikipedia Action API serially, at least 2 s apart, with `maxlag=5`, retries after HTTP 429 and a descriptive User-Agent. It was interrupted three times when its worker restarted (after 287, 318 and 359 saved responses); each time `--resume` replayed the saved responses through the same request code to rebuild URLs and continuation tokens, failed on any divergence, and sent only missing requests. The collection holds 362 responses, retrieved between 05:55 and 10:48 UTC on 2026-11-05. The vital frame is the link table of *Wikipedia:Vital articles/Level 3* at revision 1382205991 (999 titles); the random frame is MediaWiki's `page_random`. Only revision metadata (IDs, parents, timestamps, SHA-1 values, sizes) was requested.

Of 300 sampled pages, 16 are excluded as the protocol requires: 12 vital pages contain revisions whose SHA-1 is hidden, and 4 random pages have no visible revision by the end of the window. The 284 remaining pages (138 vital, 146 random) give 21,483 states of positive duration (17,709 vital, 3,774 random). 134 vital and 15 random pages return to an earlier SHA-1 at least once, in 2,583 return events, with median departures of 618 s (vital) and 377 s (random). The direction of selection bias from the exclusions is unknown; the shares and page-bootstrap intervals below describe the retained histories and do not correct for it.

The replay reconstructs an edit-only timeline from the visible revision metadata, completely paginated, with revisions that share a timestamp collapsed by the preregistered one-second convention. Administrative deletion, undeletion and other availability events were not collected, so the replay evaluates certification against the reconstructed edit history, not against what an unqualified current endpoint served over time. Captures are replayed against the frozen histories; no request was sent to Wikipedia at the simulated instants.

F.2 Equal hashes that hide a change

Capture instants are hourly over the window; a pair counts when the page exists at both instants. A pair *hides a change* when its endpoint hashes are equal but a state with another SHA-1 was current for positive time in between. Table 6 gives the preregistered measurement behind Figure 2a. Equal revision IDs hid no change at any gap.

F.3 Bundle certification

The request gap is the time between consecutive request starts; every request window lasts 1 s, so at a 1 s gap consecutive windows touch, and the minute and hour gaps separate them further. Two pages is the smallest bundle, and four and eight double it. The grid varies bundle size and timing under control; it does not model the size or pacing of real crawls. Each stratum, bundle size ($n = 2, 4, 8$) and gap uses 400 bundles drawn from the pages that exist throughout the window (138 vital and 141 random; the other 5 random pages first appear during 2024) and 10 seeded start times: 4,000 runs per method and condition, 12,000 per stratum and gap. The preregistered checks are G1, that every page had its observed content at the claimed common time (for the exact test, at the single read's instant), and G2, that no twice-read page changed between its reads. Methods marked * are preregistered.

Two observations follow from the preregistered counts (Table 7). In every condition the surplus of the hash double collect over the revision-ID double collect, in certified runs, equals its number of G2 violations; it is positive in 6 of the 18 conditions and zero in the other 12. What content hashes add consists only of certificates that bridge a hidden change. And no revision-ID certificate violates G1 or G2, as Theorem 3 predicts.

F.4 First addendum: closed bridges and the schedule

`protocol-addendum.json` was written after the preregistered results were known and is labelled post hoc. `addendum.py` evaluates every method on exactly the preregistered draws (bundles, read orders, start times and read instants), so all comparisons are paired, and it first reproduces the preregistered methods count for count. The added methods are closed-bridge double collects with hashes and with revision IDs; the double-collect schedule with the center page's second read removed (same instants, certified by the exact test); the exact revisit order with content hashes; and the revisit order in which the page read first and last offers only a content hash, certified either by trusting the hash or by Theorem 3(ii). (The frozen addendum protocols were written when the paper numbered this result Theorem 2 and call it Theorem 2(ii).)

Closed bridges add exactly the runs with touching windows: at 1 s the closed double collect certifies the same runs as the exact test, and the added certificates bring no violation. Over all conditions content hashes then issue 69,970 certificates and revision IDs 69,720, and under both schedules the hash surplus equals the hash certificates that break continuity: 250 with the double collect and 199 with the revisit order. The exact revision-ID rule never certifies fewer runs than the closed double collect, more in 9 of 18 conditions, mostly where pages change at the gap. For eight vital pages an hour apart the gain from 2,539 to 2,690 splits into 29 runs from no longer requiring the center page to stay unchanged between two reads (same instants) and 122 from the shorter schedule, whose second half starts one slot earlier. One hash page at the ends, trusted as if sound, breaks continuity 64 times; Theorem 3(ii) certifies none of these runs, because neither of its windows lies inside the other pages' common interval.

F.5 Second addendum: what a certificate asserts

`protocol-addendum-2.json` was written after the first addendum and before `addendum2.py` ran; the script first reproduces all 162 rows of the first addendum. Because the preregistered G1 is existential over the claimed interval for double collects but evaluates the single read for the exact test, it does not measure one assertion across methods. For every certified run the second addendum checks three nested assertions:

Table 6: Equal-hash revisit pairs that hid a change, with 95% bootstrap intervals over pages (2,000 resamples, seed 7).

Stratum	Gap	Pairs	Equal hash	Hidden	Share, % (95% CI)	Pages
vital	1 min	1,212,192	1,211,925	4	<0.01 (<0.01–<0.01)	4 of 138
vital	1 h	1,212,054	1,202,086	999	0.08 (0.07–0.10)	118 of 138
vital	1 d	1,208,880	1,060,423	27,445	2.59 (2.14–3.09)	127 of 138
vital	7 d	1,189,008	611,897	81,712	13.35 (11.04–15.80)	129 of 138
random	1 min	1,263,461	1,263,409	1	<0.01 (<0.01–<0.01)	1 of 146
random	1 h	1,263,315	1,262,110	41	<0.01 (<0.01–<0.01)	12 of 146
random	1 d	1,259,957	1,246,560	859	0.07 (0.02–0.15)	12 of 146
random	7 d	1,238,933	1,172,040	3,298	0.28 (0.10–0.52)	12 of 146

Table 7: Preregistered methods: certified runs and runs violating G1 and G2. Rows "all n" total 12,000 runs over $n = 2, 4, 8$; the last three rows split vital pages at 1 h by bundle size (4,000 runs each).

Condition	Content hash, $2n$, half-open			Revision ID, $2n$, half-open			Revision ID, exact, $2n-1$		
	Certified	G1	G2	Certified	G1	G2	Certified	G1	G2
vital, 1 s, all n	0	0	0	0	0	0	11,997	0	0
vital, 1 min, all n	11,929	0	5	11,924	0	0	11,941	0	0
vital, 1 h, all n	10,195	8	241	9,954	0	0	10,279	0	0
random, 1 s, all n	0	0	0	0	0	0	12,000	0	0
random, 1 min, all n	11,997	0	0	11,997	0	0	11,997	0	0
random, 1 h, all n	11,852	0	4	11,848	0	0	11,869	0	0
vital, 1 h, $n = 2$	3,882	1	14	3,868	0	0	3,930	0	0
vital, 1 h, $n = 4$	3,606	0	59	3,547	0	0	3,659	0	0
vital, 1 h, $n = 8$	2,707	7	168	2,539	0	0	2,690	0	0

Table 8: Certified runs per stratum and request gap (of 12,000), first addendum. Revision-ID double collect → center read once → revisit order is the schedule ablation behind the +29 and +122 of Figure 2b.

Stratum	Gap	Double collect		Revisit order			Hash page at the ends	
		hash	ID	ID, center once	ID	hash	trusted	Theorem 3
vital	1 s	11,997	11,997	11,997	11,997	11,997	11,997	0
vital	1 min	11,929	11,924	11,927	11,941	11,945	11,942	0
vital	1 h	10,195	9,954	10,048	10,279	10,470	10,341	0
random	1 s	12,000	12,000	12,000	12,000	12,000	12,000	0
random	1 min	11,997	11,997	11,997	11,997	11,997	11,997	0
random	1 h	11,852	11,848	11,851	11,869	11,873	11,870	0

- *continuity*: every twice-read page keeps its observed content between its first and last true reads (G2);
- *claim*: what the certificate's proof asserts on the true history, namely that every page has its observed content throughout the closed claimed interval $[\max e^-, \min s^+]$ of a double collect, or at the true read instant of an exact test's single read;
- *existence*: the combination held at some instant between the first request's start and the last request's end; a certificate that fails it is *fictitious*.

The script asserts on every run that a fictitious certificate breaks its claim and that a broken claim implies a continuity violation. It also adds the field-aware schedule of Corollary 4: the hash-only page of the off-center method moves to the center, the order becomes $order[1:] + [order[0]]$, and the revisit order is read at the preregistered slots. Pages then occupy different slots than in the all-revision-ID revisit order, so this method is paired with the others by bundle, start time and slot instants, not by page-to-slot assignment.

No certificate of any method is fictitious within the sampled runs and the reconstructed histories (Table 9); the captured combination occurs somewhere in the capture period. The median departures quoted above do not by themselves explain this zero, nor do they imply that fictitious certificates cannot occur: the MinIO execution of Figure 1a is one, since its combination held at no instant. Of the 250 hash certificates that broke continuity in the closed double collect, 56 asserted an interval in which some page had other content and 8 an interval in which the combination never held, although it held elsewhere in the capture. Revision IDs break no assertion under any schedule, and moving the hash-only page to

Table 9: The three nested assertions per method: certified runs and runs that break continuity, their claim, the preregistered G1, and existence (fictitious). Left: totals over both strata, $n = 2, 4, 8$ and the three gaps. Right: eight vital pages with request starts 1 h apart, the condition of Figure 2b. *Preregistered.

Method	All conditions, 72,000 runs					Eight vital pages, 1 h, 4,000 runs				
	Certified	Continuity	Claim	G1	Fictitious	Certified	Continuity	Claim	G1	Fictitious
content hash, $2n$, half-open*	45,973	250	56	8	0	2,707	168	33	7	0
revision ID, $2n$, half-open*	45,723	0	0	0	0	2,539	0	0	0	0
revision ID, exact, $2n-1$*	70,083	0	0	0	0	2,690	0	0	0	0
content hash, $2n$, closed	69,970	250	56	8	0	2,707	168	33	7	0
revision ID, $2n$, closed	69,720	0	0	0	0	2,539	0	0	0	0
revision ID, center read once	69,820	0	0	0	0	2,568	0	0	0	0
content hash, exact, $2n-1$	70,282	199	12	12	0	2,839	149	10	10	0
hash page at the ends, trusted	70,147	64	1	1	0	2,726	36	1	1	0
hash page at the ends, Theorem 3	0	0	0	0	0	0	0	0	0	0
hash page at the center, Theorem 3	70,032	0	0	0	0	2,653	0	0	0	0

Table 10: The hash-only page read once at the center, against the all-revision-ID revisit order on the same bundles and start times (certified of 4,000; the first method breaks no assertion anywhere).

Stratum	n	1 s	1 min	1 h
vital	2	4,000 / 4,000	3,998 / 3,999	3,936 / 3,930
vital	4	4,000 / 4,000	3,990 / 3,989	3,639 / 3,659
vital	8	3,999 / 3,997	3,948 / 3,953	2,653 / 2,690
random	2	4,000 / 4,000	4,000 / 4,000	3,998 / 3,996
random	4	4,000 / 4,000	4,000 / 4,000	3,980 / 3,980
random	8	4,000 / 4,000	3,997 / 3,997	3,894 / 3,893

the center restores 70,032 of 72,000 runs to certification, against 0 when the same page is read first and last (Table 10).

F.6 Raw-response binding

`derive.py` checks the SHA-256 of every raw API response, rebuilds every history from the raw history responses and compares page identities and revisions with the manifest, and checks the predecessor revisions. The request and continuation chains are checked at collection time instead: `collect.py--resume` replays the saved responses through the same request code and stops at any divergence. For a page without a revision inside the window, the initial state comes from a separate raw response (38 such responses). An earlier loader checked that response's hash but took the initial state from the manifest's copy without comparing the two, so editing the manifest alone could change a page's initial state while every raw file stayed intact; with the old loader such an edit to the record of *Frederick Goldie* (`raw/00324-before-150.json`) was accepted. The loader now binds that state to its raw response, checking page identity, response index, cardinality and coverage, and rejects any difference; `tests/test_revision_bundle_before_window.py` keeps a valid fixture and rejects a manifest-only edit. After the fix all eight revision-analysis outputs are byte-identical to the archived ones, and the analysis copy in `figure-artifacts/06-revision-histories/` is the same file. No reported count changed.

F.7 Scope

The frozen responses give visible revision metadata and SHA-1 values, not rendered HTML, templates, caches, serving times or administrative logs, and timestamps have one-second resolution. The vital frame is curated and the random frame follows `page_rando m`; neither samples the Web, and the shares describe the retained histories of these frames. Revision IDs advance along the reconstructed timeline; administrative deletion and undeletion, which can re-expose a preserved revision ID after absence or after another revision (Section E), are outside the replay. Their absence from the reconstruction is a limit of scope, not evidence that they did not occur.

G Controlled HTTP executions

`scripts/run_stack_experiments.py` runs six configurations, with the versions retained in each trace: Express 5.3.0 (`etag` 1.8.1) with weak and strong ETags; Python's `http.server` validating through `Last-Modified`, with advancing and restored modification times; and Apache with `FileETagMTimeSize`, with advancing and restored modification times. Every server binds only loopback. The two pages move through the phases (eligible, 100), (ineligible, 10) and (eligible, 100); the target pair (eligible, 10) is read by capturing admission in the first phase and fee twice in the middle phase, then admission again. All four reads are full GETs whose bodies are compared byte for byte with the targets; only then does a fifth request send admission's original `If-None-Match`, or `If-Modified-Since` without an ETag. A bodyless 304 never substitutes for reading the endpoint body.

The two files do not update atomically as a pair, so the order of the single-file updates matters. The forward phase replaces admission before fee, withdrawing eligibility before introducing the lower fee; restoration replaces fee before admission. Each update writes a staged file and renames it, and both pages are read back after every replacement; the trace asserts from these readbacks that the target pair was never served. Restoring admission first would briefly serve the target pair, and the run would no longer show non-coexistence. Fresh modification times advance by 10 s per phase;

Table 11: The six HTTP configurations (macOS) and the Linux re-execution.

Configuration	Repeats	Status	Pair served	Linux
Express, weak ETag	yes	304	no	same
Express, strong ETag	yes	304	no	same
Python, advancing mtime	no	200	no	same
Python, restored mtime	yes	304	no	same
Apache, advancing mtime	no	200	no	same
Apache, restored mtime	yes	304	no	same

in the restored cases the final time equals the original, an explicit mechanism condition rather than a claim about deployments.

In four configurations the endpoint validator repeats and the conditional GET returns 304 although the target pair never coexisted (Table 11). These outcomes accuse no server of violating HTTP: a 304 says that a stored copy may be reused [8]. They test the consumer's inference from equal validators to continuity, and 4 of 6 is not a failure rate. On 2026-11-05 the six configurations were re-executed on Linux x86_64 (Express 5.3.0, Python 3.13.18, Ubuntu Apache 2.4.58) with identical outcomes; the runner refuses to overwrite traces and aborts on a failed start or a violated state assertion.

H Reproduction

The derivations, checks and tests use only the Python standard library and run offline on the frozen inputs; from the repository root:

```
python3 -m unittest discover -s tests
python3 scripts/check_certification.py
python3 scripts/run_contracts.py
python3 -B experiments/reactivation_probes/derive.py
python3 -B experiments/revision_bundles/derive.py
python3 -B experiments/revision_bundles/addendum.py
python3 -B experiments/revision_bundles/addendum2.py
python3 scripts/paper_numbers.py
python3 scripts/supplement_tables.py
```

`paper_numbers.py` writes `paper/numbers.tex`, from which every number in the paper is typeset, and asserts identities among the derived files; `supplement_tables.py` writes this document's tables and number macros from the same files. Re-executing the probes or the collection needs the pinned servers or network access and produces new IDs and times; the derivations then compare classifications, not bytes.

Each directory under `figure-artifacts/` and `table-artifacts/` holds its frozen inputs, derivation, drawing or table code, exact plotted values, caption, metadata, outputs and a SHA-256 manifest, and rebuilds alone. Figures 1 and 2 are TikZ and PGFPlots code generated by `plot.py` from `plot-data.csv` and typeset with XeLaTeX in the acmart typefaces at the column width; `plot.py` checks page width, font embedding, type sizes and clipping, and rebuilds are byte-identical. This supplement is built with XeLaTeX from TeX Live 2023:

```
python3 scripts/build_figure_artifacts.py
python3 scripts/build_supplement.py
```

The artifact that accompanies the submission holds the code, the frozen data, the figure and table directories, this supplement with its sources, and `paper/numbers.tex` with every number printed in the paper. It was checked in a fresh copy before release: the tests pass, `paper_numbers.py` and `supplement_tables.py` reproduce their outputs unchanged, and `build_supplement.py` rebuilds this document byte for byte.

Each kind of evidence certifies a bounded statement. The contract checker certifies properties of the supplied graph and scope; the certification code decides the stated models; local executions show what one pinned implementation did in one run; documentation states intended service behavior; and the replay evaluates certificates against reconstructed edit histories of one wiki. None of them measures how often a deployed archive makes the inference the paper examines.

References

[1] Y. Afek, H. Attiya, D. Dolev, E. Gafni, M. Merritt, and N. Shavit. 1993. Atomic Snapshots of Shared Memory. *J. ACM* 40(4), 873–890.

[2] Z. Aghazadeh and P. Woelfel. 2015. On the Time and Space Complexity of ABA Prevention and Detection. In *PODC*, 193–202.

[3] Amazon Web Services. 2026. S3 User Guide: Object Lock considerations. https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lock-managing.html

[4] Amazon Web Services. 2026. S3 User Guide: Restoring previous versions; Managing delete markers. https://docs.aws.amazon.com/AmazonS3/latest/userguide/RestoringPreviousVersions.html

[5] Apache OpenDAL. 2026. RFC 8147: Write if not changed. https://opendal.apache.org/docs/rust/opendal/docs/rfcs/rfc_8147_write_if_not_changed/index.html

[6] C. M. Chase and V. K. Garg. 1998. Detection of Global Predicates: Techniques and Their Limitations. *Distributed Computing* 11(4), 191–201.

[7] R. Cooper and K. Marzullo. 1991. Consistent Detection of Global Predicates. In *ACM/ONR Workshop on Parallel and Distributed Debugging*, 167–174.

[8] R. Fielding, M. Nottingham, and J. Reschke. 2022. HTTP Semantics. RFC 9110.

[9] V. K. Garg and B. Waldecker. 1996. Detection of Strong Unstable Predicates in Distributed Programs. *IEEE TPDS* 7(12), 1323–1333.

[10] Google. 2026. google-cloud-storage 3.17.1 (Python), `blob.py`, lines 921–962 at commit b106af457fda1a5dadd10d5a8a52d77bf987f2ff.

[11] Google Cloud. 2026. Cloud Storage: Object metadata, generation numbers. https://docs.cloud.google.com/storage/docs/metadata

[12] Google Cloud. 2026. Cloud Storage JSON API: Objects resource. https://docs.cloud.google.com/storage/docs/json_api/v1/objects

[13] Google Cloud. 2026. Cloud Storage: Object Versioning; Objects: restore. https://docs.cloud.google.com/storage/docs/object-versioning

[14] MediaWiki. 2026. Manual: Revision table; Help: Deletion and undeletion. https://www.mediawiki.org/wiki/Manual:Revision_table

[15] Microsoft. 2026. Snapshot Blob; Get Blob (Azure Storage REST API). https://learn.microsoft.com/en-us/rest/api/storageservices/snapshot-blob

[16] Microsoft. 2026. Blob versioning in Azure Storage. https://learn.microsoft.com/en-us/azure/storage/blobs/versioning-overview

[17] M. Spaniol, D. Denev, A. Mazeika, P. Senellart, and G. Weikum. 2009. Data Quality in Web Archiving. In *WICOW*, 19–26.

[18] S. D. Stoller. 2000. Detecting Global Predicates in Distributed Systems with Clocks. *Distributed Computing* 13(2), 85–98.

[19] N. Sulimany, T. Cory, and E. Petrank. 2026. Adaptive Snapshots Require Visible Reads. In *DISC*, 1:1–1:24.

[20] A. Tarafdar and V. K. Garg. 1998. Addressing False Causality while Detecting Predicates in Distributed Programs. In *ICDCS*, 94–101.

[21] WordPress. 2026. Code reference: `WP_REST_Posts_Controller::prepare_links`, `wp_get_latest_revision_id_and_total_count`, `wp_restore_post_revision`. https://developer.wordpress.org/reference/

[22] J. Yin, L. Alvisi, M. Dahlin, and A. Iyengar. 2001. Engineering Server-Driven Consistency for Large Scale Dynamic Web Services. In *WWW*, 45–57.